



# 대규모 시스템 설계 10장

## Ch10 - 알림 시스템 설계 (발표)

### 10장. 알림 시스템 설계 (Design a Notification System)

- 알림 유형
- 전체 아키텍처
  - 연락처 정보 수집
  - 초기 설계와 한계
  - 개선된 설계
  - 컴포넌트별 역할
- 상세 설계
  - 신뢰성 (재시도, 중복 제거)
  - 알림 템플릿, 수신 설정
  - 발송 한도 제어
  - 모니터링과 이벤트 추적

## Part 1.

### 1. 개략적 설계

#### 알림 유형

| 유형     | 전달 경로                 | 대표 서비스     |
|--------|-----------------------|------------|
| iOS 푸시 | Provider → APNs → iOS | Apple APNs |

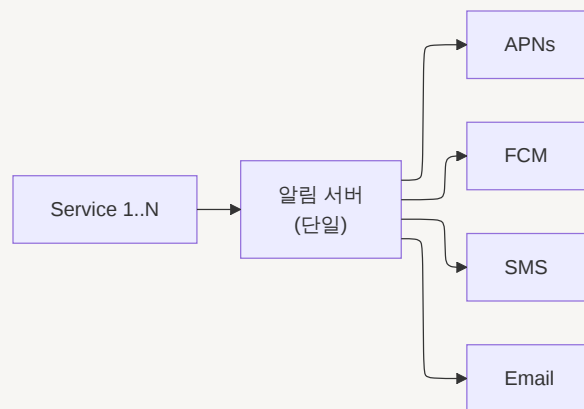
| 유형         | 전달 경로                          | 대표 서비스              |
|------------|--------------------------------|---------------------|
| Android 푸시 | Provider → FCM → Android       | Firebase FCM        |
| SMS        | Provider → SMS Gateway → 휴대폰   | Twilio, Nexmo       |
| 이메일        | Provider → Email Service → 수신함 | SendGrid, Mailchimp |

- 알림 서버는 단말에 직접 전송하지 않음. 모든 채널이 **서드파티를 경유**
- 채널마다 프로토콜, 인증, 장애 특성이 전부 다름

## 연락처 정보 수집

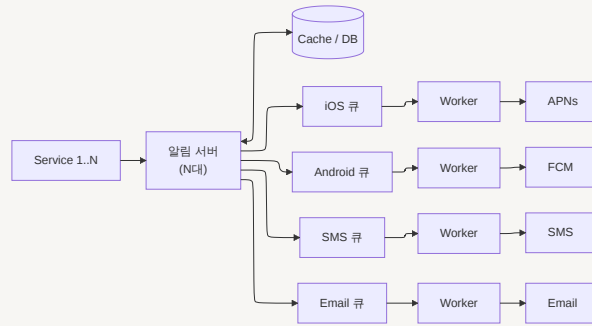
- 앱 설치, 가입 시점에 API 서버가 수집
- users 테이블 (email, phone 등)과 devices 테이블 (device\_token)
- user : device = **1:N** (한 사용자가 여러 기기 사용)

## 초기 설계 (문제 있는 버전)



| 문제            | 원인                                 |
|---------------|------------------------------------|
| 단일 장애점 (SPOF) | 알림 서버가 한 대                         |
| 확장성 없음        | 수평 확장 불가                           |
| 성능 병목         | DB 조회, 템플릿, 외부 API 호출을 한 서버가 전부 처리 |

## 개선된 설계



### 핵심 변경 3가지

1. DB, 캐시를 알림 서버에서 분리
2. 알림 서버 수평 확장 및 로드밸런싱
3. 채널별 메시지 큐 및 Worker 도입

### 컴포넌트별 역할

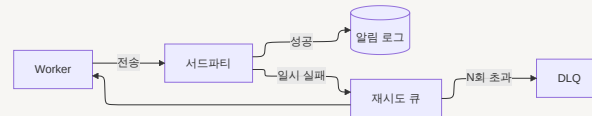
| 한국어 명칭   | 영문                  | 역할                              |
|----------|---------------------|---------------------------------|
| 알림 서버    | Notification Server | API 수신, 검증, 데이터 조회, 큐 적재        |
| 메시지 큐    | Message Queue       | 채널별로 분리. 알림 서버와 Worker를 비동기로 연결 |
| 작업 서버    | Worker              | 큐에서 메시지를 꺼내 서드파티에 전송            |
| 알림 로그 DB | Notification Log    | 전송 이력 기록. 장애 시 복구에 사용           |
| 재시도 큐    | Retry Queue         | 전송 실패한 메시지를 재처리                 |
| 템플릿      | Template            | 반복 구조를 미리 정의해 일관성 유지            |
| 수신 설정    | Preference          | 사용자별 채널, 주제 수신 동의 여부 저장         |
| 한도 제어    | Rate Limiter        | 사용자당, 서비스당 발송 한도 제어             |

## 2. 상세 설계

### 2-1. 신뢰성 : 재시도와 중복 제거

"실패는 무조건 일어난다"를 전제로 설계

| 문제               | 대응                    |
|------------------|-----------------------|
| 서드파티 일시 장애       | 재시도 큐, 지수 백오프         |
| 영구 실패 (잘못된 토큰 등) | Dead Letter Queue로 이동 |
| 같은 알림 중복 전송      | 역등성 키로 중복 제거          |
| 알림 로그 손실         | 전송 전후 DB에 기록          |



## 2-2. 알림 템플릿, 수신 설정

- **템플릿**: 반복되는 포맷을 미리 정의. 매번 새로 만들지 않음
- **수신 설정**: 전송 전 반드시 확인. 사용자가 수신 거부한 채널은 건너뛰
- 수신 설정 테이블 예시

| user_id | 채널    | 수신 동의 |
|---------|-------|-------|
| 1001    | push  | true  |
| 1001    | sms   | false |
| 1001    | email | true  |

## 2-3. 발송 한도 제어 (Rate Limiting)

- 과도한 알림은 사용자 이탈의 직접적 원인
- 제한 기준: 사용자당, 서비스당, 채널당
- 초과 시 정책: 폐기, 지연 발송, 묶음 중 선택

## 2-4. 모니터링과 이벤트 추적

| 관점  | 지표                           |
|-----|------------------------------|
| 시스템 | 큐 적체량, Worker 처리량, 서드파티 응답시간 |
| 사용자 | 열람률, 클릭률, 참여율, 수신 거부 비율      |
| 비용  | 채널별 발송량 (SMS, 이메일은 건당 비용)    |

## 핵심 정리

| 속성                    | 핵심 설계 결정            | 기억할 것               |
|-----------------------|---------------------|---------------------|
| 신뢰성 (Reliability)     | 재시도 큐, DLQ, 로그 DB   | 최소 1회 전달 및 중복 제거    |
| 확장성 (Scalability)     | 채널별 큐 및 Worker 풀 분리 | 단일 장애점 제거, 수평 확장    |
| 제어성 (Controllability) | 수신 설정, 한도 제어, 템플릿   | 사용자 수신 동의 존중        |
| 관측성 (Observability)   | 채널별 지표, 이벤트 추적      | 전체 평균이 아닌 채널별로 봐야 함 |

## Part 2.

### 목차

- 알림 묶음 (Aggregation)
- 멱등성 (Idempotency)

## 1. 알림 묶음 (Aggregation)

### 문제

짧은 시간에 같은 사용자에게 다수의 알림이 발생할 때

- 메신저: 30초 동안 메시지 10개 → 진동 10번
- SNS: 좋아요 100개 → 알림 100개
- 이커머스: 장바구니 상품 알림 건별 발송

### 실제 서비스 적용 사례

| 서비스         | 적용 방식                  |
|-------------|------------------------|
| 카카오톡, Slack | "철수님에게서 새 메시지 N개"      |
| Instagram   | "영희님 외 N명이 좋아요를 눌렀습니다" |
| GitHub      | 일간 дай제스트 이메일          |
| LinkedIn    | 주간 활동 요약               |

## 설계 결정 3가지

| 결정     | 옵션                                                        | 선택 기준                        |
|--------|-----------------------------------------------------------|------------------------------|
| 시간 윈도우 | 고정 윈도우, 슬라이딩 (디바운스)                                       | 실시간성이 중요하면 디바운스, 주기적 발송이면 고정 |
| 묶음 키   | (user_id, sender_id), (user_id, post_id), (user_id, type) | 사용자가 인지하는 "한 덩어리"의 단위        |
| 합성 방식  | 개수, 최신, 요약                                                | 푸시 공간 제약에 맞춰 선택              |

## 구현 흐름



- 버퍼: Redis Sorted Set 또는 Hash
- 비우기 트리거: TTL 만료 또는 스케줄러
- 긴급 알림은 버퍼 거치지 않고 바로 푸시 큐로 우회

## 고민: 윈도우 중간에 사용자가 앱을 열었다면

- 이미 내용을 본 사용자에게 알림을 보낼 필요가 없음
- 클라이언트에서 "읽음" 이벤트를 쏘고, 서버는 해당 사용자의 버퍼를 비우지 않고 폐기
- 어느 계층에서 판단할지 (알림 서버, Worker, 클라이언트)가 회사마다 다름

## 토론

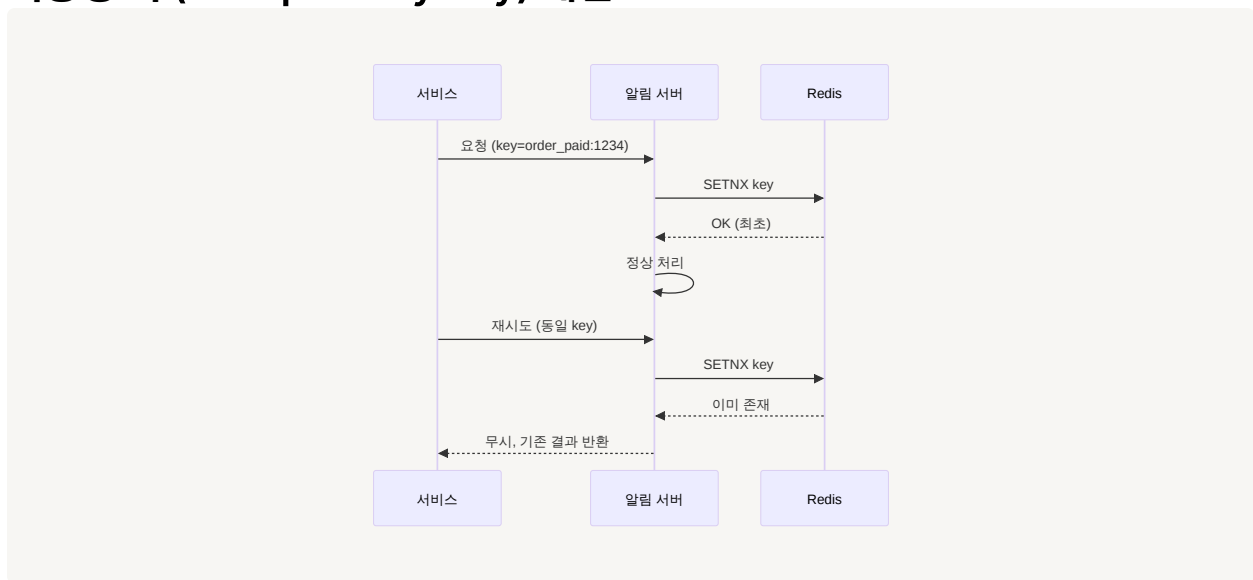
- 각자 서비스에서 알림 묶음을 실제로 적용하는 사례가 있는가. 묶음 키와 시간 윈도우를 어떤 기준으로 잡았는가
- 보안 경고나 결제 실패처럼 묶으면 안 되는 긴급 알림을 어떻게 예외 처리하는가. 알림 타입에 플래그( `aggregatable: false` )를 두는 방식과 아예 별도 전송 경로를 두는 방식 중 어느 쪽이 관리하기 좋은가
- 묶음 처리를 서버 측 버퍼링으로 할 것인가, FCM `collapse_key` 처럼 단말이 도착한 알림을 덮어쓰게 맡길 것인가. 두 방식의 차이(서버는 N개를 1개로 합성, 단말은 마지막 하나만 표시)를 어느 상황에서 어떻게 선택하는가

## 2. 멍등성 (Idempotency)

## 중복 발생 경로 3가지

| 경로       | 원인                 | 방어 위치           |
|----------|--------------------|-----------------|
| 호출 측 재시도 | 서비스가 알림 요청을 두 번 보냄 | 알림 서버 API 입구    |
| 큐 중복 소비  | MQ가 최소 1회 전달 보장    | Worker 처리 진입    |
| 서드파티 재전송 | 응답 못 받아 재호출        | Worker 전송 직후 기록 |

## 역등성 키 (Idempotency Key) 패턴



## 구현 결정

| 결정      | 옵션             | 권장                                       |
|---------|----------------|------------------------------------------|
| 키 생성 주체 | 알림 서버, 호출 서비스  | <b>호출 서비스</b> (재시도 포함 동일 키)              |
| 키 저장소   | Redis, DB, 메모리 | Redis (속도와 영속성의 균형)                      |
| TTL     | 1시간, 24시간, 7일  | 호출 측 재시도 정책에 맞춰 24시간부터 7일까지              |
| 키 네이밍   | 이벤트 기반         | <code>{event_type}:{entity_id}</code> 형식 |

## 실용적 한계

- 분산 시스템에서 **정확히 1회 전달**은 불가능
- 실무 표준: **최소 1회 전달에 중복 제거**를 얹는 방식
- SMS, 이메일은 건당 비용 발생 → 푸시보다 엄격한 기준 적용

## 직접 구현 전에 확인: 이미 나와있는 솔루션들

회사마다 멍등성 테이블을 직접 만들거나 Redis에 키를 넣는 코드를 짜는 경우가 많은데, 패턴이 비슷하다 보니 이미 라이브러리, 프레임워크, 인프라 레벨에서 제공하는 경우가 많다. 직접 구현 전에 한 번 훑어볼 가치가 있다.

| 계층          | 솔루션                                                                                                    | 특징                                     |
|-------------|--------------------------------------------------------------------------------------------------------|----------------------------------------|
| 메시지 브로커     | Kafka Idempotent Producer, SQS FIFO <code>MessageDeduplicationId</code>                                | 설정 한 줄로 활성화. 같은 브로커 내에서만 보장            |
| HTTP API 헤더 | Stripe 스타일 <code>Idempotency-Key</code> (IETF 표준화 진행 중)                                                | 결제 API의 사실상 표준. UUID 하나로 24시간 보장       |
| API Gateway | Kong, AWS API Gateway 멍등성 플러그인                                                                         | 서비스마다 구현할 필요 없이 게이트웨이에서 일괄 처리          |
| 언어/프레임워크    | Python <code>dramatiq</code> , Node <code>express-idempotency</code> , Spring <code>@Idempotent</code> | 데코레이터/애노테이션 한 줄로 적용                    |
| 스트림 처리      | Kafka Streams, Flink의 정확히 1회 전달 (exactly-once)                                                         | 프레임워크 레벨에서 트랜잭션과 묶어 보장. 외부 시스템 호출에는 한계 |

- **결제/외부 API 호출**이면 Stripe 스타일 헤더 패턴이 정답에 가깝다
- **사내 메시지 큐** 처리면 쓰는 브로커의 기본 기능을 먼저 켜본다
- **여러 서비스가 공통 규약이 필요**하면 API Gateway 레벨에서 처리
- 직접 구현은 "비즈니스 키(`order_paid:1234` 처럼 이벤트 의미가 담긴 키)를 쓰고 싶은데 기존 솔루션이 지원 안 할 때"가 사실상 유일한 이유

## 토론

- 현재 회사에서 멍등성 처리를 어떻게 하고 있는가. 직접 구현인지, 아니면 위에 정리한 솔루션 중 하나를 쓰고 있는가. 직접 구현을 선택했다면 그 이유는 무엇인가
- 사내 여러 서비스가 알림을 호출할 때 멍등성 키 규약을 강제하는 현실적인 방법이 있는가 (SDK 배포, Gateway 검증, 내용 해시 대안 등)
- 멍등성 키 TTL은 호출 측의 최대 재시도 기간 이상이어야 하는데, 여러 서비스가 각자 다른 재시도 정책을 쓸 때 TTL을 어떻게 통일할 것인가 (일괄 최대값, 서비스별 분리, API 파라미터화 등)
- 중복은 두 지점에서 발생한다. 하나는 **API 입구**(호출 측 재시도로 같은 요청이 두 번 들어옴), 다른 하나는 **Worker 진입**(큐가 같은 메시지를 두 번 전달함). 두 지점 모두에서 중복

제거를 돌릴 것인가, 아니면 입구 한 곳에서만 막고 큐를 믿을 것인가. 쓰고 있는 큐 시스템 (Kafka, SQS, RabbitMQ 등)의 전달 보장 수준에 따라 판단이 달라지는가